# DAG 최적화 및 제약 조건 단순화 분석

## 개요

DAG(Directed Acyclic Graph) 모듈은 Circom 컴파일러에서 제약 조건 생성 후 최적화 단계를 담당하는 핵심 모듈입니다. 이 모듈은 생성된 제약 조건들을 효율적인 그래프 구조로 표현하고, 다양한 최적화 기법을 적용하여 최종적으로 R1CS 형태의 제약 조건 리스트로 변환합니다.

## 7.1 의존성 그래프 구축 및 최적화 분석

### DAG 클래스의 핵심 데이터 구조

DAG 모듈의 중심에는 `DAG` 구조체가 있으며, 이는 전체 회로의 계층적 구조를 그래프 형태로 표현합니다.

```
pub struct DAG {
pub one_signal: usize, // 상수 1을 나타내는 신호 ID
pub nodes: Vec<Node>, // 템플릿 인스턴스들을 나타내는 노드들
pub adjacency: Vec<Vec<Edge>>, // 노드 간 연결 관계를 나타내는 인접 리스트
pub prime: String, // 사용되는 유한체의 소수
}
```

#### Node 구조체 - 템플릿 인스턴스 표현

각 `Node` 는 하나의 템플릿 인스턴스를 나타내며, 해당 인스턴스의 모든 정보를 포함합니다:

```rust
pub struct Node {
entry: Edge, // 진입점 정보
template_name: String, // 템플릿 이름
parameters: Vec<BigInt>, // 템플릿 매개변수
number_of_signals: usize, // 총 신호 개수
number_of_components: usize, // 하위 컴포넌트 개수
// 신호 분류별 개수
intermediates_length: usize, // 중간 신호 개수
public_inputs_length: usize, // 공개 입력 신호 개수
inputs_length: usize, // 전체 입력 신호 개수
outputs_length: usize, // 출력 신호 개수
// 신호 관리
signal_correspondence: HashMap<String, Signal>, // 신호 이름 → ID 매핑
ordered_signals: Vec<String>, // 순서가 있는 신호 목록
locals: HashSet<usize>, // 로컬 신호 집합
reachables: HashSet<usize>, // 접근 가능한 신호 집합
forbidden_if_main: HashSet<usize>, // 메인일 때 금지된 신호들
io_signals: Vec<usize>, // 입출력 신호 목록
// 제약 조건 및 최적화 정보
constraints: Vec<Constraint>, // 해당 노드의 제약 조건들
underscored_signals: Vec<usize>, // 언더스코어 신호들
is_parallel: bool, // 병렬 처리 가능 여부
```

```
has_parallel_sub_cmp: bool, // 병렬 하위 컴포넌트 존재 여부
is_custom_gate: bool, // 커스텀 게이트 여부
number_of_subcomponents_indexes: usize, // 하위 컴포넌트 인덱스 개수
}
```

### Edge 구조체 - 컴포넌트 간 연결 관계

Edge 는 부모 컴포넌트와 자식 컴포넌트 간의 연결을 나타냅니다:

```
pub struct Edge {
label: String, // 컴포넌트 인스턴스 이름
goes_to: usize, // 대상 노드 ID
in_number: usize, // 입력 신호 시작 번호
out_number: usize, // 출력 신호 끝 번호
in_component_number: usize, // 입력 컴포넌트 번호
out_component_number: usize, // 출력 컴포넌트 번호
}
```

## DAG 구축 과정

### 1. 노드 추가 ( add_node )

새로운 템플릿 인스턴스가 생성될 때마다 DAG에 노드가 추가됩니다:

```
pub fn add_node(
&mut self,
template_name: String,
```

```
parameters: Vec<BigInt>,
is_parallel: bool,
is_custom_gate: bool
) -> usize {
let id = self.nodes.len();
self.nodes.push(
Node::new(id, template_name, parameters, is_parallel, is_custom_gate)
);
self.adjacency.push(vec![]);
id
}
```

### 2. 엣지 추가 ( `add_edge` )

컴포넌트 인스턴스화 시 부모-자식 관계를 나타내는 엣지가 추가됩니다:

```
pub fn add_edge(&mut self, to: usize, label: &str, is_parallel: bool) -> Opti
if to < self.main_id() {
let from = self.main_id();
let in_num = self.nodes[from].number_of_signals;
let out_num = in_num + self.nodes[to].number_of_signals;
// 신호 번호 업데이트
self.nodes[from].number_of_signals += self.nodes[to].number_of_signals;
self.nodes[from].entry.out_number += self.nodes[to].number_of_signals;
```

```
// 병렬 처리 정보 전파
self.nodes[from].has_parallel_sub_cmp |= self.nodes[to].is_parallel || is_parall
let  with = Edge {
label: label.to_string(),
goes_to: to,
in_number: in_num,
out_number: out_num,
// ... 기타 필드들
};
// 신호 대응 관계 업데이트
self.update_signal_correspondence(from, to, &with, label);
self.adjacency[from].push(with);
self.adjacency[from].last()
} else {
Option::None
}
}
```

### 3. 신호 관리

DAG는 다양한 타입의 신호들을 체계적으로 관리합니다:

```
// 입력 신호 추가
pub  fn  add_input(&mut  self, name: String, is_public: bool) {
```

```
if let Option::Some(node) = self.get_mut_main() {
node.add_input(name, is_public);
}
}

// 출력 신호 추가
pub fn add_output(&mut self, name: String) {
if let Option::Some(node) = self.get_mut_main() {
node.add_output(name);
}
}

// 중간 신호 추가
pub fn add_intermediate(&mut self, name: String) {
if let Option::Some(node) = self.get_mut_main() {
node.add_intermediate(name);
}
}
```

## Tree 구조체 - DAG 순회를 위한 뷰

`Tree` 구조체는 DAG를 순회하면서 특정 노드의 컨텍스트를 제공합니다:

```
pub struct Tree<'a> {
```

```rust
dag: &'a DAG, // 원본 DAG 참조
pub field: BigInt, // 유한체 정보
pub path: String, // 현재 경로 (예: "main.multiplier.adder")
pub offset: usize, // 신호 번호 오프셋
pub node_id: usize, // 현재 노드 ID
pub signals: Vec<usize>, // 현재 노드의 신호들
pub forbidden: HashSet<usize>, // 금지된 신호들
pub id_to_name: HashMap<usize, String>, // 신호 ID → 이름 매핑
pub constraints: Vec<Constraint>, // 현재 노드의 제약 조건들
}
```

### Tree 생성 및 순회

```rust
impl<'a> Tree<'a> {
// 루트 트리 생성
pub fn new(dag: &DAG) -> Tree {
let constants = UsefulConstants::new(&dag.prime);
let field = constants.get_p().clone();
let root = dag.get_main().unwrap();
let node_id = dag.main_id();
let offset = dag.get_entry().unwrap().in_number;
// ... 초기화 로직
}
```

```
// 서브트리로 이동
pub fn go_to_subtree(current: &'a Tree, edge: &Edge) -> Tree<'a> {
let field = current.field.clone();
let dag = current.dag;
let node_id = edge.goes_to;
let node = &current.dag.nodes[node_id];
let path = format!("{}.{}", current.path, edge.label);
let offset = current.offset + edge.in_number;
// ... 서브트리 구성 로직
}
}
```

## map_to_list 함수 - DAG를 ConstraintList로 변환

`map_to_list` 함수는 DAG 구조를 평면적인 제약 조건 리스트로 변환하는 핵심 함수입니다:

```
pub fn map_to_list(self, flags: SimplificationFlags) -> ConstraintList {
map_to_constraint_list::map(self, flags)
}
```

### map 함수의 상세 구현

`map_to_constraint_list::map` 함수는 다음과 같은 단계로 진행됩니다:

```
pub fn map(dag: DAG, flags: SimplificationFlags) -> ConstraintList {
```

```
let constants = UsefulConstants::new(&dag.prime);

let field = constants.get_p().clone();

let init_id = dag.main_id();

let no_public_inputs = dag.public_inputs();

let no_public_outputs = dag.public_outputs();

let no_private_inputs = dag.private_inputs();

// 1. 금지된 신호 집합 초기화

let mut forbidden = dag.get_main().unwrap().forbidden_if_main.clone();

// 2. 제약 조건 분류를 위한 홀더 생성

let mut c_holder = CHolder::default();

// 3. 신호 매핑 벡터 초기화

let mut signal_map = vec![0];

// 4. 트리 순회를 통한 제약 조건 수집

let no_constraints = map_tree(&Tree::new(&dag), &mut signal_map, &mut c_holde

let max_signal = Vec::len(&signal_map);

// 5. DAG 인코딩 생성

let name_encoding = produce_encoding(no_constraints, init_id, dag.nodes, dag.ad

// 6. Simplifier 생성 및 최적화 실행

Simplifier {

field,

no_public_inputs,

no_public_outputs,

no_private_inputs,
```

```
forbidden,
max_signal,
dag_encoding: name_encoding,
linear: c_holder.linear,
equalities: c_holder.equalities,
cons_equalities: c_holder.constant_equalities,
no_rounds: flags.no_rounds,
flag_s: flags.flag_s,
parallel_flag: flags.parallel_flag,
flag_old_heuristics: flags.flag_old_heuristics,
port_substitution: flags.port_substitution,
json_substitutions: flags.json_substitutions,
}
.simplify_constraints()
}
```

### map_tree 함수 - 재귀적 트리 순회

```
fn  map_tree(
tree: &Tree,
witness: &mut Vec<usize>,
c_holder: &mut CHolder,
forbidden: &mut HashSet<usize>
```

```
) -> usize {
let mut no_constraints = 0;

// 1. 현재 노드의 신호들을 witness에 추가
for signal in &tree.signals {
Vec::push(witness, *signal);
if tree.dag.nodes[tree.node_id].is_custom_gate {
forbidden.insert(*signal);
}
}

// 2. 제약 조건들을 타입별로 분류
for constraint in &tree.constraints {
if Constraint::is_constant_equality(constraint) {
LinkedList::push_back(&mut c_holder.constant_equalities, constraint.clone());
} else if Constraint::is_equality(constraint, &tree.field) {
LinkedList::push_back(&mut c_holder.equalities, constraint.clone());
} else if Constraint::is_linear(constraint) {
LinkedList::push_back(&mut c_holder.linear, constraint.clone());
} else {
no_constraints += 1;
}
}
```

```
// 3. 자식 노드들을 재귀적으로 처리
for edge in Tree::get_edges(tree) {
let subtree = Tree::go_to_subtree(tree, edge);
no_constraints += map_tree(&subtree, witness, c_holder, forbidden);
}
no_constraints
}
```

## SimplificationFlags - 최적화 플래그 시스템

`SimplificationFlags` 구조체는 다양한 최적화 옵션을 제어합니다:

```
pub struct SimplificationFlags {
pub no_rounds: usize, // 최적화 라운드 수
pub flag_s: bool, // 단순화 플래그 (false면 선형 최적화 적용)
pub parallel_flag: bool, // 병렬 처리 플래그
pub port_substitution: bool, // 치환 포팅 플래그
pub json_substitutions: String, // JSON 치환 로그 파일 경로
pub flag_old_heuristics: bool, // 구 휴리스틱 사용 플래그
pub prime: String, // 유한체 소수
}
```

### 최적화 플래그별 동작

1. **no_rounds**: 반복적 최적화 라운드 수를 제어

- 0: 최적화 없음
- 1 이상: 지정된 횟수만큼 최적화 반복

2. **flag_s**: 선형 제약 조건 최적화 제어

- true: 선형 최적화 건너뛰기
- false: 클러스터 기반 선형 최적화 적용

3. **parallel_flag**: 병렬 처리 활성화

- 클러스터 단순화 시 멀티스레딩 사용

4. **flag_old_heuristics**: 최적화 알고리즘 선택

- true: 기존 휴리스틱 사용
- false: 새로운 최적화 알고리즘 사용

## 병렬 처리 최적화

DAG 모듈은 대규모 회로의 효율적 처리를 위해 병렬 처리를 지원합니다:

### 1. 노드 레벨 병렬성

```
impl Node {
pub fn has_parallel_sub_cmp(&self) -> bool {
self.has_parallel_sub_cmp
}
}
```

### 2. 클러스터 기반 병렬 최적화

제약 조건 단순화 시 독립적인 클러스터들을 병렬로 처리:

```
// constraint_simplification.rs에서
let pool = ThreadPool::new(num_cpus::get());
for cluster in clusters {
let cluster_tx = cluster_tx.clone();
let job = move || {
let result = eq_cluster_simplification(cluster, &forbidden, &field);
cluster_tx.send((id, result)).unwrap();
};
ThreadPool::execute(&pool, job);
}
```

## DAG 인코딩 시스템

DAG를 ConstraintList로 변환할 때 사용되는 인코딩 시스템:

```
pub struct DAGEncoding {
pub init: usize, // 초기 노드 ID
pub no_constraints: usize, // 비선형 제약 조건 수
pub nodes: Vec<EncodingNode>, // 인코딩된 노드들
pub adjacency: Vec<Vec<EncodingEdge>>, // 인코딩된 엣지들
}

pub struct EncodingNode {
```

```
pub id: usize, // 노드 ID
pub name: String, // 템플릿 이름
pub parameters: Vec<BigInt>, // 매개변수들
pub signals: Vec<SignalInfo>, // 신호 정보들
pub ordered_signals: Vec<usize>, // 순서가 있는 신호들
pub non_linear: LinkedList<C>, // 비선형 제약 조건들
pub is_custom_gate: bool, // 커스텀 게이트 여부
}
```

이러한 DAG 구조와 최적화 시스템을 통해 Circom 컴파일러는 복잡한 계층적 회로를 효율적으로 처리하고 최적화된 제약 조건을 생성할 수 있습니다.

## 7.2 제

약 조건 단순화 알고리즘 분석

### simplification_process 함수의 전체 구조

`constraint_simplification.rs` 의 `simplification` 함수는 Circom 컴파일러의 핵심 최적화 엔진입니다. 이 함수는 여러 단계의 최적화를 순차적으로 적용하여 제약 조건의 수를 최소화하고 증명 생성 효율성을 극대화합니다.

```
pub fn simplification(smp: &mut Simplifier) -> (ConstraintStorage, SignalMap,
// 1. 초기화 및 설정
let mut substitution_log = if smp.port_substitution {
Some(SubstitutionJSON::new(&smp.json_substitutions).unwrap())
} else {
```

```
None

};

let  apply_linear = !smp.flag_s;

let  use_old_heuristics = smp.flag_old_heuristics;

let  field = smp.field.clone();

let  forbidden = Arc::new(std::mem::replace(&mut  smp.forbidden, HashSet::with_c

// 2. 관련 신호 집합 구축

let  relevant_signals = build_relevant_set();

// 3. 단일 할당 단순화 (Equality Simplification)

let  single_substitutions = eq_simplification();

// 4. 상수 할당 단순화 (Constant Equality Simplification)

let  cons_substitutions = constant_eq_simplification();

// 5. 선형 제약 조건 단순화 (Linear Simplification)

let  linear_substitutions = linear_simplification();

// 6. 비선형 제약 조건 처리

let (with_linear, mut  constraint_storage) = build_constraint_storage();

// 7. 반복적 최적화 라운드

iterative_optimization_rounds();

// 8. 증인 재구축

let  signal_map = rebuild_witness();

(constraint_storage, signal_map, private_inputs_witness)

}
```

### 1. 관련 신호 집합 구축 (Relevant Signal Set Building)

최적화 과정에서 실제로 사용되는 신호들만을 식별하여 불필요한 계산을 방지합니다:

```
fn build_relevant_set(
mut iter: EncodingIterator,
relevant: &mut HashSet<usize>,
renames: &SEncoded,
deletes: &SEncoded,
) {
// 현재 노드의 비선형 제약 조건에서 사용되는 신호들 수집
let (_, non_linear) = EncodingIterator::take(&mut iter);
for c in non_linear {
for signal in C::take_cloned_signals(&c) {
let signal = unwrapped_signal(renames, signal).unwrap_or(signal);
if !SEncoded::contains_key(deletes, &signal) {
HashSet::insert(relevant, signal);
}
}
}

// 자식 노드들을 재귀적으로 처리
for edge in EncodingIterator::edges(&iter) {
let next = EncodingIterator::next(&iter, edge);
```

```
build_relevant_set(next, relevant, renames, deletes)

}

}
```

## 2. 등식 단순화 (Equality Simplification)

신호 간의 등식 관계를 이용한 치환을 통해 제약 조건을 단순화합니다.

### 클러스터 구축 알고리즘

```
fn build_clusters(linear: LinkedList<C>, no_vars: usize) -> Vec<Cluster> {

let mut arena = ClusterArena::with_capacity(no_linear);

let mut cluster_to_current = ClusterPath::with_capacity(no_linear);

let mut signal_to_cluster = vec![no_linear; no_vars];

for constraint in linear {

if !constraint.is_empty() {

let signals = C::take_cloned_signals(&constraint);

let dest = ClusterArena::len(&arena);

// 새 클러스터 생성

ClusterArena::push(&mut arena, Some(Cluster::new(constraint, signals.len())));

Vec::push(&mut cluster_to_current, dest);

// 신호들을 클러스터에 연결하고 기존 클러스터와 병합

for signal in signals {

let prev = signal_to_cluster[signal];

signal_to_cluster[signal] = dest;
```

```
if prev < no_linear {
arena_merge(&mut arena, &mut cluster_to_current, prev, dest);
}
}
}
}
// 유효한 클러스터들만 반환
let mut clusters = Vec::new();
for cluster in arena {
if let Some(cluster) = cluster {
if Cluster::size(&cluster) != 0 {
Vec::push(&mut clusters, cluster);
}
}
}
clusters
}
```

### 등식 클러스터 단순화

```
fn eq_cluster_simplification(
mut cluster: Cluster,
forbidden: &HashSet<usize>,
```

```
field: &BigInt,
) -> (LinkedList<S>, LinkedList<C>) {
if Cluster::size(&cluster) == 1 {
// 단일 등식 처리: s1 = s2 형태
let constraint = LinkedList::pop_back(&mut cluster.constraints).unwrap();
let signals: Vec<_> = C::take_cloned_signals_ordered(&constraint).iter().cloned
let s_0 = signals[0];
let s_1 = signals[1];
if HashSet::contains(forbidden, &s_0) && HashSet::contains(forbidden, &s_1) {
// 둘 다 금지된 신호면 제약 조건 유지
LinkedList::push_back(&mut constraints, constraint);
} else if HashSet::contains(forbidden, &s_0) {
// s_0가 금지되면 s_1 = s_0로 치환
LinkedList::push_back(&mut substitutions, S::new(s_1, A::Signal { symbol: s_0 }
} else if HashSet::contains(forbidden, &s_1) {
// s_1이 금지되면 s_0 = s_1로 치환
LinkedList::push_back(&mut substitutions, S::new(s_0, A::Signal { symbol: s_1 }
} else {
// 둘 다 자유로우면 큰 번호를 작은 번호로 치환
let (l, r) = if s_0 > s_1 { (s_0, s_1) } else { (s_1, s_0) };
LinkedList::push_back(&mut substitutions, S::new(l, A::Signal { symbol: r }).un
}
} else {
```

```
// 다중 등식 처리: 여러 신호가 같은 값을 가지는 경우
let (mut remains, mut min_remains) = (BTreeSet::new(), None);
let (mut remove, mut min_remove) = (HashSet::new(), None);
// 금지된 신호와 자유로운 신호 분류
for c in cluster.constraints {
for signal in C::take_cloned_signals_ordered(&c) {
if HashSet::contains(&forbidden, &signal) {
BTreeSet::insert(&mut remains, signal);
min_remains = Some(min_remains.map_or(signal, |s| std::cmp::min(s, signal)));
} else {
min_remove = Some(min_remove.map_or(signal, |s| std::cmp::min(s, signal)));
HashSet::insert(&mut remove, signal);
}
}
}

// 대표 신호 선택 (금지된 신호 우선, 없으면 가장 작은 번호)
let rh_signal = if let Some(signal) = min_remains {
BTreeSet::remove(&mut remains, &signal);
signal
} else {
let signal = min_remove.unwrap();
HashSet::remove(&mut remove, &signal);
```

```
signal
};

// 나머지 금지된 신호들에 대한 제약 조건 생성
for signal in remains {
let l = A::Signal { symbol: signal };
let r = A::Signal { symbol: rh_signal };
let expr = A::sub(&l, &r, field);
let c = A::transform_expression_to_constraint_form(expr, field).unwrap();
LinkedList::push_back(&mut cons, c);
}

// 자유로운 신호들에 대한 치환 생성
for signal in remove {
let sub = S::new(signal, A::Signal { symbol: rh_signal }).unwrap();
LinkedList::push_back(&mut subs, sub);
}
}
(substitutions, constraints)
}
```

### 3. 상수 등식 단순화 (Constant Equality Simplification)

신호와 상수 간의 등식을 처리합니다:

```
fn constant_eq_simplification(
c_eq: LinkedList<C>,
forbidden: &HashSet<usize>,
field: &BigInt,
substitution_log: &mut Option<SubstitutionJSON>,
) -> (LinkedList<S>, LinkedList<C>) {
let mut cons = LinkedList::new();
let mut subs = LinkedList::new();
for constraint in c_eq {
let mut signals: Vec<_> = C::take_cloned_signals_ordered(&constraint).iter().c
let signal = signals.pop().unwrap();
if HashSet::contains(&forbidden, &signal) {
// 금지된 신호면 제약 조건 유지
LinkedList::push_back(&mut cons, constraint);
} else {
// 신호를 상수값으로 치환
let sub = C::clear_signal_from_linear(constraint, &signal, field);
LinkedList::push_back(&mut subs, sub);
}
}
log_substitutions(&subs, substitution_log);
```

```
(subs, cons)

}
```

## 4. 선형 제약 조건 단순화 (Linear Simplification)

가장 복잡한 최적화 단계로, 클러스터 기반 병렬 처리를 통해 선형 제약 조건들을 최적화합니다:

```
fn linear_simplification(

log: &mut Option<SubstitutionJSON>,

linear: LinkedList<C>,

forbidden: Arc<HashSet<usize>>,

no_labels: usize,

field: &BigInt,

use_old_heuristics: bool,

) -> (LinkedList<S>, LinkedList<C>) {

let mut cons = LinkedList::new();

let mut substitutions = LinkedList::new();

// 1. 클러스터 구축

let clusters = build_clusters(linear, no_labels);

// 2. 병렬 처리를 위한 스레드 풀 생성

let (cluster_tx, simplified_rx) = mpsc::channel();

let pool = ThreadPool::new(num_cpus::get());

let no_clusters = Vec::len(&clusters);

// 3. 각 클러스터를 병렬로 처리
```

```
for cluster in clusters {
let cluster_tx = cluster_tx.clone();
let config = Config {
field: field.clone(),
constraints: cluster.constraints,
forbidden: Arc::clone(&forbidden),
num_signals: cluster.num_signals,
use_old_heuristics,
};
let job = move || {
let result = full_simplification(config);
cluster_tx.send(result).unwrap();
};
ThreadPool::execute(&pool, job);
}
// 4. 결과 수집
ThreadPool::join(&pool);
for _ in 0..no_clusters {
let mut result = simplified_rx.recv().unwrap();
log_substitutions(&result.substitutions, log);
LinkedList::append(&mut cons, &mut result.constraints);
LinkedList::append(&mut substitutions, &mut result.substitutions);
}
```

```
(substitutions, cons)
}
```

## 5. 비선형 제약 조건 처리

비선형 제약 조건들을 ConstraintStorage에 저장하고 치환을 적용합니다:

```
pub fn obtain_and_simplify_non_linear(
mut iter: EncodingIterator,
storage: &mut ConstraintStorage,
frames: &SFrames,
field: &BigInt,
) -> LinkedList<C> {
let mut linear = LinkedList::new();
// 현재 노드의 비선형 제약 조건 처리
let (_, non_linear) = EncodingIterator::take(&mut iter);
for mut constraint in non_linear {
// 모든 치환 프레임 적용
for frame in frames {
fast_encoded_constraint_substitution(&mut constraint, frame, &field);
}
C::fix_constraint(&mut constraint, &field);
// 치환 후 선형이 되었는지 확인
if C::is_linear(&constraint) {
```

```
linear.push_back(constraint);
} else {
storage.add_constraint(constraint);
}
}
// 자식 노드들 재귀 처리
for edge in EncodingIterator::edges(&iter) {
let next = EncodingIterator::next(&iter, edge);
let mut linear_in_next = obtain_and_simplify_non_linear(next, storage, frames,
linear.append(&mut linear_in_next);
}
linear
}
```

## 6. 반복적 최적화 라운드

여러 라운드에 걸쳐 최적화를 반복하여 더 많은 단순화를 달성합니다:

```
let mut round_id = 0;
let mut linear = with_linear;
let mut apply_round = apply_linear && no_rounds > 0 && !linear.is_empty();
let mut non_linear_map = build_non_linear_signal_map(&constraint_storage);

while apply_round {
```

```
// 1. 선형 제약 조건 단순화
let (substitutions, mut constants) = linear_simplification(
&mut substitution_log,
linear,
Arc::clone(&forbidden),
no_labels,
&field,
use_old_heuristics,
);


// 2. 삭제된 신호 추적
for sub in &substitutions {
deleted.insert(*sub.from());
}
// 3. 상수 제약 조건에 치환 적용
lconst.append(&mut constants);
for constraint in &mut lconst {
for substitution in &substitutions {
C::apply_substitution(constraint, substitution, &field);
}
C::fix_constraint(constraint, &field);
}
// 4. 비선형 제약 조건에 치환 적용하여 새로운 선형 제약 조건 생성
```

```
linear = apply_substitution_to_map(
&mut constraint_storage,
&mut non_linear_map,
&substitutions,
&field,
);
round_id += 1;
no_rounds -= 1;
apply_round = !linear.is_empty() && no_rounds > 0;
}
```

## 7. 증인 재구축 (Witness Rebuilding)

최적화 과정에서 삭제된 신호들을 제거하고 새로운 신호 매핑을 생성합니다:

```
fn rebuild_witness(
max_signal: usize,
deleted: &mut HashSet<usize>,
forbidden: &HashSet<usize>,
non_linear_map: SignalToConstraints,
remove_unused: bool,
) -> SignalMap {
let mut map = SignalMap::with_capacity(max_signal);
let mut free = LinkedList::new();
```

```
for signal in 0..max_signal {
if deleted.contains(&signal) {
// 삭제된 신호는 자유 슬롯으로 추가
free.push_back(signal);
} else if remove_unused && !forbidden.contains(&signal) && !non_linear_map.con
// 사용되지 않는 신호 제거
deleted.insert(signal);
free.push_back(signal);
} else if let Some(new_pos) = free.pop_front() {
// 자유 슬롯에 신호 재배치
map.insert(signal, new_pos);
free.push_back(signal);
} else {
// 원래 위치 유지
map.insert(signal, signal);
}
}
map
}
```

## 8. produce_witness 함수 - 증인 생성 과정

DAG 구조를 순회하면서 증인 벡터를 생성합니다:

```rust
pub fn produce_witness(dag: &DAG) -> Vec<usize> {
let mut witness = vec![0]; // 첫 번째 원소는 항상 0 (상수 1의 값)
let tree = Tree::new(dag);
produce_tree_witness(&tree, &mut witness);
Vec::shrink_to_fit(&mut witness);
witness
}

fn produce_tree_witness(tree: &Tree, witness: &mut Vec<usize>) {
// 현재 노드의 모든 신호를 증인에 추가
for signal in &tree.signals {
Vec::push(witness, *signal);
}
// 자식 노드들을 재귀적으로 처리
for edge in Tree::get_edges(tree) {
let subtree = Tree::go_to_subtree(tree, edge);
produce_tree_witness(&subtree, witness);
}
}
```

## 최적화 기법 요약

1. **죽은 코드 제거 (Dead Code Elimination)**

- 사용되지 않는 신호들을 식별하고 제거
- `remove_unused` 플래그로 제어

2. **상수 전파 (Constant Propagation)**

- 상수값으로 확정된 신호들을 치환으로 제거
- `constant_eq_simplification` 에서 처리

3. **제약 조건 병합 (Constraint Merging)**

- 동일한 신호를 공유하는 제약 조건들을 클러스터로 묶어 최적화
- `build_clusters` 함수에서 Union-Find 알고리즘 사용

4. **병렬 처리 최적화**

- 독립적인 클러스터들을 멀티스레드로 병렬 처리
- `ThreadPool` 을 사용한 작업 분산

5. **반복적 최적화**

- 여러 라운드에 걸쳐 최적화를 반복
- 각 라운드에서 새로운 최적화 기회 발견

이러한 포괄적인 최적화 시스템을 통해 Circom 컴파일러는 복잡한 회로도 효율적인 R1CS 제약 조건으로 변환할 수 있습니다.